



# РСХБ ЦИФРА

**32K+**

Охват за 30 дней

## РСХБ.Цифра (Россельхозбанк)

Меняем банк и сельское хозяйство

4,29 Оценка работодателя

178,13 Рейтинг

34 813 Подписчики

[Подписаться](#)**ydergach**

21 минуту назад

## Как организовать балансировку нагрузки Backend приложений Java Spring Cloud + Kubernetes

🕒 5 мин 👁 618

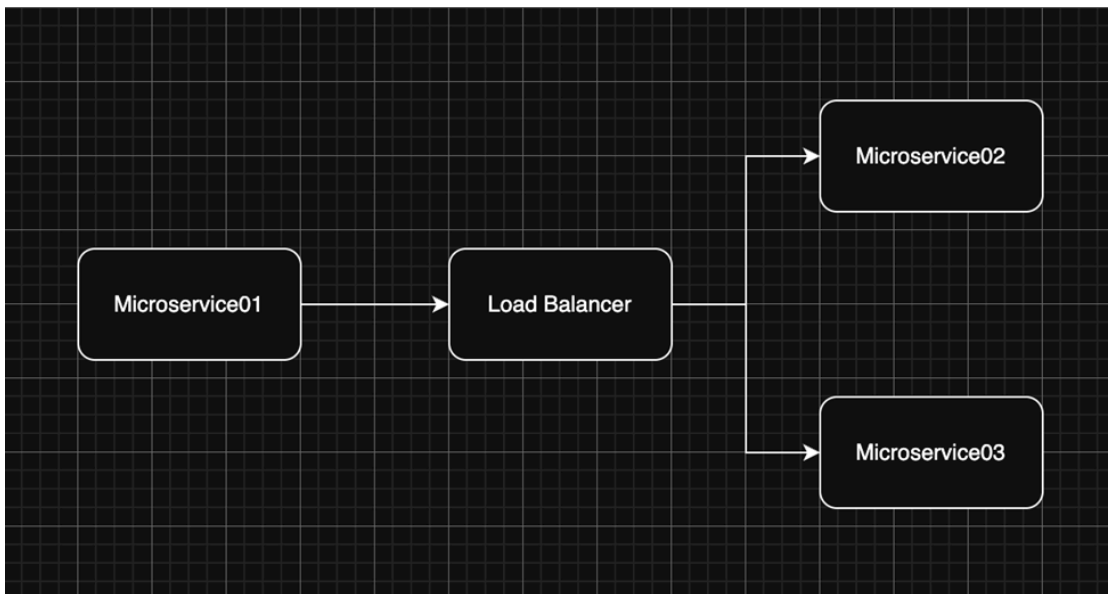
Блог компании РСХБ.Цифра (Россельхозбанк), Микросервисы\*, Kubernetes\*, Java\*, Open source\*

Привет, Хабр! Я Юрий Дергач, я возглавляю ЦК DevOps и релизного управления в РСХБ. Мы с командой развиваем инфраструктуру и автоматизируем разработку продуктов компании. При внедрении наших проектов группы «Экосистема Свое», частично основанных на стеке Java Spring, в Kubernetes, возникли вопросы, связанные с различными методами балансировки нагрузки между микросервисами.



В этой статье мы обсудим два основных подхода к балансировке нагрузки между Backend-компонентами приложений на стеке Java Spring Cloud в Kubernetes. Мы также рассмотрим преимущества и недостатки каждого метода.

## Server-side Load Balancing



Server-side Load Balancing — это, как правило, стандартная (нативная) модель балансировки трафика по средствам штатных средств среды kubernetes. В роли балансировщика выступает сущность Kubernetes Service.

Kubernetes Service имеет 3 режима работы:

1) iptables - Стандартный режим (рекомендован до Kubernetes v1.26). Использует цепочки iptables. Алгоритм балансировки - Случайный выбор (random)

2) ipvs - Режим на основе IP Virtual Server (рекомендован с v1.26+).

Поддерживает 6 алгоритмов:

- rr (round-robin)
- lc (least connection)
- dh (destination hashing)
- sh (source hashing)
- sed (shortest expected delay)
- nq (never queue)

3) userspace - Устаревший режим (не рекомендуется).

Можно построить свою (кастомную) балансировку внутри кластера Kubernetes по средствам Istio, или популярного HAProxy. Но, как правило, такие подходы используются для решения специфичных задач, например построение балансировки с шифрованием трафика или использования безопасных контуров информационных систем с продвинутым фаерволом.

## Client-side Load Balancing

Client-side Load Balancing — это подход, при котором клиент сам выбирает сервис из списка доступных экземпляров для отправки запроса, минуя центральный балансировщик. В Kubernetes это часто применяется в микросервисных архитектурах для повышения эффективности и снижения задержек.

### Как работает Client-side Load Balancing в Kubernetes?

Клиент получает список всех доступных подов сервиса через:

- DNS-запросы (например, через headless-сервис: nslookup [my-svc.my-namespace.svc.cluster.local](#)).
- Kubernetes API (используя клиентские библиотеки, например, Java-приложение с kubernetes-client).
- Специализированные инструменты (Consul, Eureka, Zookeeper).

Выбор алгоритма балансировки.

Клиентская библиотека применяет алгоритм для выбора пода:

- Round Robin
- Random
- Weighted
- Least Connections
- Zone-aware (учёт зоны доступности)

Запрос идёт напрямую в контейнер, минуя kube-proxy.

Рассмотрим подробнее подход Client-side Load Balancing на примере Spring Cloud Kubernetes.

Spring Cloud Kubernetes интегрирует экосистему Spring Cloud с Kubernetes, используя нативные возможности платформы для реализации паттернов микросервисной архитектуры. Основные принципы работы:

### 1. Service Discovery (обнаружение сервисов)

- Используется Kubernetes API.

Принцип работы:

- Каждый сервис регистрируется в Kubernetes как Service (с типом ClusterIP).
- Spring Cloud Kubernetes обращается к Kubernetes API для получения списка доступных сервисов.
- Клиенты используют DNS-имена вида {service-name}.{namespace}.svc.cluster.local.

### 2. Load Balancing (балансировка нагрузки)

- Spring Cloud Kubernetes использует Spring Cloud LoadBalancer для L7-балансировки на стороне клиента.

Принцип работы:

- Клиент получает список IP-адресов подов через DiscoveryClient.
- Логика балансировки (Round Robin, Random и т.д.) применяется на стороне клиента.

### 3. Distributed Tracing (распределенная трассировка)

- Интеграция с Jaeger/Zipkin через стандартные инструменты Spring Cloud Sleuth.

#### 4. Service Resilience (устойчивость сервисов)

Используются стандартные Spring Cloud компоненты:

- Hystrix или Resilience4j для Circuit Breaker.
- Ribbon (устаревший) или Spring Cloud LoadBalancer.

Ключевые компоненты:

1. spring-cloud-starter-kubernetes-client:

- Основная зависимость для доступа к Kubernetes API.
- Автоматически настраивает DiscoveryClient

2. spring-cloud-starter-kubernetes-ribbon (опционально):

- Интеграция с Ribbon для клиентской балансировки (в новых версиях заменена на Spring Cloud LoadBalancer).

Если говорить простым языком, Spring Cloud Kubernetes позволяет сервисам внутри Kubernetes общаться «напрямую» между собой, и использовать встроенные механизмы для обеспечения балансировки, трассировки и отказоустойчивости вне зависимости от namespaces.

Подведем итоги и рассмотрим плюсы и минусы подходов по балансировке нагрузки.

#### Server-side Load Balancing (балансировка нагрузки на стороне сервера) в Kubernetes

Плюсы:

- **Централизованное управление.** В Kubernetes можно централизованно управлять настройками балансировки через API и инструменты оркестрации. Это упрощает мониторинг и

управление распределением нагрузки.

- **Высокая производительность.** Kubernetes может оптимизировать распределение нагрузки между подинстансами (pods) для обеспечения максимальной эффективности и балансировки ресурсов.
- **Надёжность.** В случае сбоя одного из подинстансов Kubernetes может автоматически перераспределить трафик на другие доступные подинстансы, обеспечивая высокую доступность приложений.
- **Возможность использования сложных алгоритмов балансировки.** Kubernetes поддерживает использование различных алгоритмов балансировки, включая те, которые оптимизируют распределение нагрузки на основе различных параметров.

#### Минусы:

- **Зависимость от инфраструктуры.** Для реализации server-side load balancing в Kubernetes требуется мощная инфраструктура и ресурсы кластера.
- **Задержки.** Централизованное управление и обработка запросов могут привести к небольшим задержкам из-за необходимости координации между компонентами системы.
- **Сложность настройки.** Настройка и оптимизация алгоритмов балансировки в Kubernetes может быть сложной задачей, требующей глубоких знаний и опыта.

## Client-side Load Balancing (балансировка нагрузки на стороне клиента) в Kubernetes

#### Плюсы:

- **Снижение нагрузки на сервер.** В Kubernetes можно использовать клиентские балансировщики (например, Spring Cloud Kubernetes), что снижает нагрузку на компоненты кластера Kubernetes.
- **Уменьшение задержек.** Поскольку балансировка может происходить на стороне клиента, это может снизить общую задержку при обращении к приложениям в Kubernetes.
- **Гибкость.** Клиенты могут самостоятельно выбирать серверы или подинстансы для подключения, что обеспечивает гибкость в распределении нагрузки.
- **Возможность использования простых алгоритмов балансировки.** На стороне клиента можно использовать более простые алгоритмы, что упрощает реализацию и настройку.

#### Минусы:

- **Необходимость в дополнительных настройках на стороне клиента.** Пользователи должны настроить параметры балансировки на своих клиентах, что может потребовать дополнительных усилий и времени.
- **Сложность управления.** Управление распределением нагрузки может быть менее централизованным и более сложным для мониторинга в крупных кластерах.
- **Возможные проблемы с равномерным распределением нагрузки.** Клиенты могут выбирать серверы неравномерно, что может привести к неравномерной нагрузке на подинстансы в

Kubernetes.

- **Ограниченная возможность использования сложных алгоритмов.** На стороне клиента сложнее реализовать сложные алгоритмы балансировки, которые могут потребоваться для оптимизации распределения нагрузки в сложных сценариях.

Сначала мы выбрали Server-side Load Balancing из-за его простоты для команд разработки. Однако уже рассматриваем более продвинутую балансировку на основе Service Discovery, чтобы решать более сложные задачи.

**Теги:** [микросервисы](#), [kubernetes](#), [java spring](#)

**Хабы:** [Блог компании РСХБ.Цифра \(Россельхозбанк\)](#), [Микросервисы](#), [Kubernetes](#), [Java](#), [Open source](#)



### Редакторский дайджест

Присылаем лучшие статьи раз в месяц

Подписаться

Оставляя почту, я принимаю [Политику конфиденциальности](#) и даю согласие на получение рассылок



32K+

Охват за 30 дней

### РСХБ.Цифра (Россельхозбанк)

Меняем банк и сельское хозяйство

[Сайт](#) [ВКонтакте](#) [Telegram](#) [Сайт](#) [Telegram](#) [Сайт](#) [Сайт](#)



2

Карма

**Дергач Юрий** [@ydergach](#)

IT engineer

Подписаться



[Комментировать](#)

### Публикации

[ЛУЧШИЕ ЗА СУТКИ](#) [ПОХОЖИЕ](#)



**the\_bat**

23 часа назад

**Ремонт блока питания с Power Delivery. 470 граммов электроники**

 Простой  4 мин  16K

 +96

 30

 27

 **interpres**

18 часов назад

## Электроинструмент становится хуже, и это делается намеренно

 Простой  11 мин  26K

Обзор

Перевод

 +68

 45

 81

 **prohronus**

18 часов назад

## Как ИИ-агенты стали новым оружием скамеров на Хабр Карьере

 5 мин  9.2K

Кейс

 +51

 9

 9

 **alizar**

22 часа назад

## Готовимся к отключению. Эффективные форматы для упаковки и раздачи HTML-страниц

 Простой  7 мин  12K

Обзор

 +37

 59

 18

 **beget\_com**

23 часа назад

## Механический калькулятор. Как работает арифмометр?

 Средний  10 мин  9.5K

Обзор

 +37

 23

 8

 **TrexSelectel**

19 часов назад

## Linux 7.0 и что изменилось в ядре после очередного цикла разработки

 5 мин  8.7K

Обзор

 +27

 10

 0

 **AlexSerbul**

23 часа назад

## Программирование с AI-ассистентом — похороните меня под плинтусом



Средний

14 мин

9.9K

Мнение

+26

33

11

mariklolik

18 часов назад

Как переложить нагрузку по code review с разработчиков на LLM

Средний

7 мин

7.4K

Кейс

+23

23

6

nick\_dyuba

22 часа назад

Легенды 90-х — кто придумал и производил жвачки Turbo, Love is... и ТіріТір

5 мин

7K

Recovery Mode

+22

8

5

Mimizavr

23 часа назад

«Великое очищение» в работе с контентом: что осталось от роли редактора

11 мин

6.9K

+17

9

7

Показать еще

ИНФОРМАЦИЯ

|                  |                                                            |
|------------------|------------------------------------------------------------|
| Сайт             | <a href="http://www.rshbdigital.ru">www.rshbdigital.ru</a> |
| Дата регистрации | 20 августа 2020                                            |
| Дата основания   | 15 марта 2000                                              |
| Численность      | свыше 10 000 человек                                       |
| Местоположение   | Россия                                                     |

ССЫЛКИ

[РСХБ в цифре](#)  
[rshbdigital.ru](http://rshbdigital.ru)

В КОНТАКТЕ

**РСХБ.Цифра**

808 followers



Follow on VK

ПРИЛОЖЕНИЯ

**Своё Родное**

Маркетплейс натуральных фермерских продуктов и услуг в сфере агротуризма

[Android](#) [iOS](#)

ВИДЖЕТ



**Календарь  
ИТ-событий**  
Конференции, хакатоны,  
митапы для всех уровней

БЛОГ НА ХАБРЕ

21 минуту назад

Как организовать балансировку нагрузки Backend приложений Java Spring Cloud + Kubernetes

618 0

14 апр в 11:10

Тренды в обучении ИТ-специалистов в России: Как импортозамещение и новые вызовы меняют образовательный ландшафт

6.7K 4

13 апр в 17:04

ИТ-право: обзор нового законопроекта о регулировании ИИ

5K 7

7 апр в 14:46

Тест-драйв от RS-Lab: обзор импортозамещённого ноутбука KVADRA LE14U

9.9K 21

1 апр в 11:09

Что общего у вина и ИТ?

6.4K 6

Ваш аккаунт

Войти  
Регистрация

Разделы

Статьи  
Новости  
Хабы  
Компании  
Авторы  
Песочница

Информация

Устройство сайта  
Для авторов  
Для компаний  
Документы  
Соглашение  
Конфиденциальность

Услуги

Корпоративный блог  
Медийная реклама  
Нативные проекты  
Образовательные программы  
Стартапам



Настройка языка

Техническая поддержка

© 2006–2026, Habr